

Real-Time RAG (Retrieval Augmented Generation) Application for Movie Review using Apache Kafka

Ali Charara

Rémy Sader

January 2025

Abstract

This report presents the design and implementation of a real-time Retrieval-Augmented Generation (RAG) system for analyzing live movie reviews. The system ingests streaming data from the TMDB API using Apache Kafka, generates semantic embeddings with the Gemini model, and stores them in a vector database for fast retrieval. Users can query the system to obtain context-aware summaries of the latest reviews, combining both recency and relevance. The project demonstrates how RAG can be effectively applied to continuously updating data streams, providing timely insights and interactive analysis.

Contents

1	Introduction	3
2	Project Objectives	3
3	System Architecture	3
3.1	Data Ingestion	4
3.2	Data Processing and Embeddings	4
3.3	Vector Database	4
3.4	RAG Query and Generation	4
4	Implementation	5
4.1	Kafka and TMDb API	6
4.2	Kafka Producer Implementation	7
4.3	Kafka Consumer	8
4.4	Vector Database Integration	8
4.5	Embedding Generation Using the Gemini Model	8
4.6	RAG Pipeline with LangChain	9
4.7	Interactive Web Interface	10
5	Examples Prompt and Response	10
6	Conclusion	11

1 Introduction

Retrieval-Augmented Generation (RAG) systems combine large language models with external knowledge sources to provide contextually accurate responses. While most RAG systems operate on static datasets, real-time applications require processing continuously updated streaming data, introducing challenges in high-throughput ingestion and low-latency retrieval.

This project focuses on a **real-time RAG system** for analyzing public opinions on newly released movies. Using Apache Kafka, the system ingests live reviews from the **TMDB (The Movie Database) API**, generates embeddings, and stores them in a vector database. When a user queries a movie, the system retrieves the most recent and relevant reviews to produce an up-to-date summary of public sentiment, highlighting key aspects such as plot, cinematography, and actor performances.

This work demonstrates how RAG can be applied to streaming data, providing timely insights while leveraging Kafka and TMDB for efficient real-time data processing.

2 Project Objectives

The main objective of this project is to develop a **real-time RAG system** capable of analyzing and summarizing public opinions on newly released movies. The specific goals are as follows:

- Implement a data ingestion pipeline using **Apache Kafka** to collect live movie reviews from the **TMDB API**.
- Process and generate embeddings for streaming reviews, and continuously update a **vector database** for low-latency retrieval.
- Retrieve the most recent and relevant reviews based on user queries to provide accurate, context-aware summaries.
- Demonstrate the ability of a RAG system to operate effectively in a real-time streaming environment.

These objectives emphasize both the **technical aspects** of real-time data processing and the **practical application** of RAG for movie review analysis.

3 System Architecture

The system architecture consists of several components designed to support real-time retrieval and generation of movie review summaries. Figure 1 shows the overall pipeline.

3.1 Data Ingestion

Live movie reviews are collected from the **TMDB API** using **Apache Kafka**. Kafka acts as a message broker, allowing the system to continuously receive and buffer streaming data. This ensures that incoming reviews are available for processing as soon as they arrive.

3.2 Data Processing and Embeddings

Each review is preprocessed, including cleaning and tokenization, before being converted into a numerical representation called an **embedding**. These embeddings capture the semantic meaning of the reviews and are used for similarity search.

3.3 Vector Database

Embeddings are stored in a **vector database**, which is optimized for fast retrieval of similar items. The database is continuously updated as new reviews arrive, enabling the system to always access the most recent information.

3.4 RAG Query and Generation

When a user queries the system about a movie, the RAG model retrieves the most relevant and recent reviews from the vector database. The retrieved context is then used to generate an up-to-date summary, highlighting key aspects such as plot, cinematography, and actor performances.

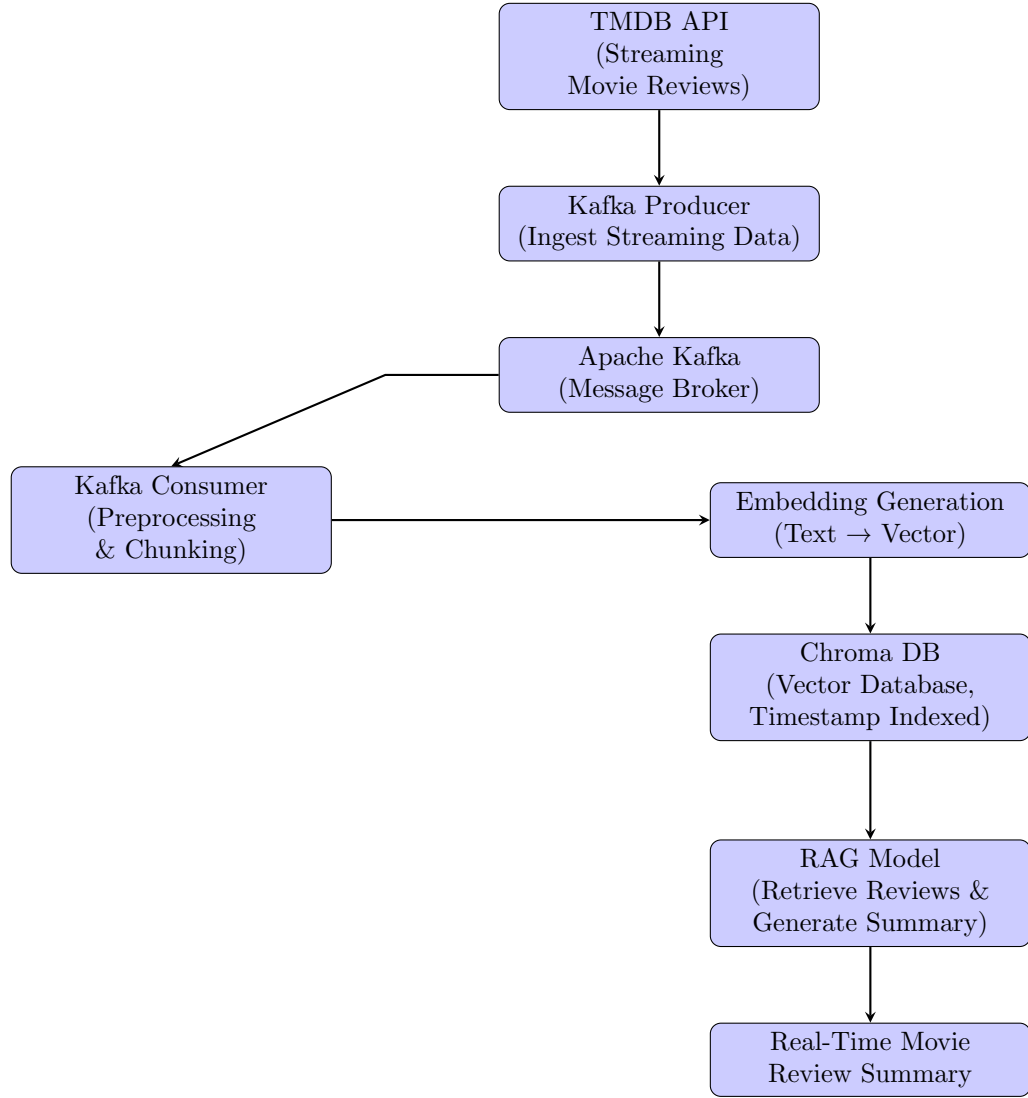


Figure 1: Overview of the real-time RAG system architecture.

This architecture ensures that the system can handle live movie reviews efficiently and provide timely, context-aware summaries.

4 Implementation

The implementation of the real-time RAG system involves several components. In this section, we first describe the data ingestion pipeline using **Apache**

Kafka and the **TMDB API**, followed by other components such as embedding generation and vector database integration.

4.1 Kafka and TMDB API

To collect live movie data, the system uses the **TMDB (The Movie Database) API**, which provides real-time information on movies, including metadata, popularity metrics, and user reviews. TMDB is suitable for this project because it offers structured and up-to-date information on current, popular, top-rated, and upcoming movies.

Two Kafka **topics** were created to organize the data stream:

```
bin/kafka-topics.sh --create --topic movie-reviews \
--bootstrap-server localhost:9092
```

```
bin/kafka-topics.sh --create --topic movie-details \
--bootstrap-server localhost:9092
```

A topic in Kafka acts as a communication channel. Producers publish messages to a topic, and consumers subscribe to it to receive data. In our system:

- **movie-details** stores metadata about each movie.
- **movie-reviews** stores user reviews linked to these movies.

TMDB endpoints used:

- `/movie/now_playing`
- `/movie/popular`
- `/movie/top_rated`
- `/movie/upcoming`
- `/movie/{id}/reviews`
- `/movie/{id}`

For each endpoint, the producer processes at most 50 movies per iteration to avoid flooding the pipeline. From the movie details response, only the metadata relevant to our project is selected, such as:

- **Movie details:** ID, title, overview, release date, runtime, genres, vote average, popularity.
- **Review content:** author, review body, timestamp, movie ID, movie title.

Each review message is enriched with metadata (movie ID, title, timestamp) before being sent to Kafka. This structure ensures that reviews can be efficiently retrieved and matched with their corresponding movies during the RAG retrieval and embedding stages later in the pipeline.

4.2 Kafka Producer Implementation

The Kafka producer continuously fetches movie information and user reviews from TMDB and publishes the data to the corresponding topics. It represents the entry point of the pipeline, ensuring that the remainder of the architecture always works with fresh, real-time data.

From each TMDB category (now playing, popular, top rated, upcoming), the producer retrieves a list of movies and selects up to 50 entries. For each movie:

1. The API `/movie/{id}` is called to obtain movie metadata.
2. The result is formatted as a structured JSON payload and sent to the `movie-details` topic.
3. The API `/movie/{id}/reviews` is then called to collect up to 100 reviews.
4. Each review is enriched with custom fields (`movie_id`, `movie_title`, and `timestamp`) and sent to the `movie-reviews` topic.

To avoid sending duplicates, a set of processed movie IDs is kept in memory. This prevents movie metadata from being republished multiple times.

Finally, the producer runs in a continuous loop with a small delay between iterations, ensuring real-time ingestion without overwhelming the API or the Kafka cluster.

Example of data fetched by the Kafka producer:

Field	Value
Movie ID	1131759
Title	Omniscient Reader: The Prophecy
Overview	Kim Dok-ja, an ordinary man in his 20s, is the only reader of an obscure web novel titled "Three Ways to Survive the Apocalypse". After having read the last chapter, the novel suddenly becomes reality, and its omnipotent hero Yu Jung-hyeok appears before Kim. As the only person who knows how to survive in this world, Kim and his companions strive to save the world by writing his own, new ending.
Release Date	2025-07-23
Runtime	117
Genres	Action, Adventure, Fantasy
Vote Average	6.671
Popularity	274.4064

Table 1: Example of movie metadata retrieved by the producer.

Field	Value
Author	IMDB
Username	IMDBofficiellaccount
Avatar Path	/tem9LYNDSgxmj3pYjbz6xROM7ZX.png
Rating	10.0
Content	My Review of FNAF 2... (truncated)
Created At	2025-12-08T16:28:20.181Z
Review ID	6936fca491e3d4c0ebf7c891
Movie ID	1228246
Movie Title	Five Nights at Freddy’s 2
Timestamp	1767302536.21011

Table 2: Example of a movie review retrieved by the producer.

4.3 Kafka Consumer

The Kafka consumer reads movie reviews from the `movie-reviews` topic, deserializes each message, and prepares it for storage in the vector database. The consumer now also performs lightweight data cleaning to ensure consistency before generating embeddings.

Key processing steps:

- Assigns a unique identifier to each review (fallback if missing).
- Normalizes metadata such as movie title, author, rating, and timestamps.
- Converts review dates into a usable numeric timestamp for filtering.

The consumer generates embeddings using the Gemini model `text-embedding-004` (768 dimensions).

The processed content, embedding vector, and essential metadata are stored in a new ChromaDB instance named `chroma.db_v2`, which serves as the knowledge base for downstream retrieval in the RAG pipeline. This ensures clean, structured, and ready-to-query data for real-time question answering.

4.4 Vector Database Integration

The processed review content, embeddings generated by the Gemini model, and essential metadata are stored in a persistent **ChromaDB** collection (`chroma.db_v2`). This database serves as the knowledge base for the RAG pipeline, ensuring that all data is structured, queryable, and ready for real-time retrieval. Only the metadata relevant for retrieval (such as movie title, rating, and review timestamp) is included, keeping storage efficient while maintaining semantic accuracy.

4.5 Embedding Generation Using the Gemini Model

The textual content of each review is converted into a numerical vector embedding using the *Gemini* embedding model. For our project, we use the

`text-embedding-004` model, which produces embeddings of dimensionality 768. We chose Gemini for several reasons:

- High-quality semantic embeddings suitable for retrieval-augmented tasks.
- Robust handling of streaming data with consistent performance.
- Flexibility in configuring embedding parameters, such as `task_type` (like "RETRIEVAL_DOCUMENT" or "RETRIEVAL_QUERY") and embedding dimensionality.

A custom `GeminiHelper` class is used to interact with the Gemini API. It provides:

- Automatic rotation of API keys to handle quota limits.
- Easy methods to generate embeddings for single reviews or batches.
- Configurable task types and embedding dimensions to optimize semantic retrieval.

The embeddings are stored in the vector database along with the review content and relevant metadata, allowing efficient similarity-based retrieval. This forms the backbone of the RAG pipeline, enabling real-time query responses based on the semantic meaning of the reviews.

4.6 RAG Pipeline with LangChain

The real-time RAG service integrates both movie reviews and descriptions into a LangChain-based pipeline to answer user queries. This component is responsible for:

- Processing user questions and extracting structured filters (like movie title, rating ranges, or time windows) using the Gemini model.
- Retrieving relevant movie reviews and descriptions from ChromaDB, either via semantic similarity searches using embeddings or metadata-only filtering.
- Formatting the retrieved documents into a context suitable for the LLM to generate answers.
- Producing structured output in JSON format, including the final answer and referenced movies.

The pipeline uses custom `GeminiEmbeddings` and `GeminiLLM` wrappers to interface with the Gemini API for both embedding generation and LLM calls. Retrieved reviews and descriptions are ranked by relevance or timestamp, ensuring that the most pertinent information is presented to the model. The final system allows real-time, structured question answering over the movie knowledge base with semantic understanding and filter-based precision.

4.7 Interactive Web Interface

A Streamlit-based web application was developed to allow users to interact with the RAG system in real-time. Users can type questions about movies, and the system retrieves relevant reviews from ChromaDB and generates context-aware answers using the Gemini model. The interface includes live feedback, metadata inspection, and minimalistic styling to enhance usability.

5 Examples Prompt and Response

To illustrate the real-time RAG system in action, a user can ask a question about a specific movie. For instance, the following prompts were submitted:

“What do people think about the new action movie ‘Stellar Siege’?”
“Give me and action movie rated above 7 and give me a small description”

The RAG pipeline extracted relevant filters (movie title, recent timeframe, ratings if applicable), retrieved the most relevant reviews from the ChromaDB vector database, and generated a contextualized answer. The output summarizes current public opinions on the film, highlighting key aspects such as plot, actor performances, and visual effects.

Figure 2 shows an example of this interaction, with the user prompt and the RAG-generated response.

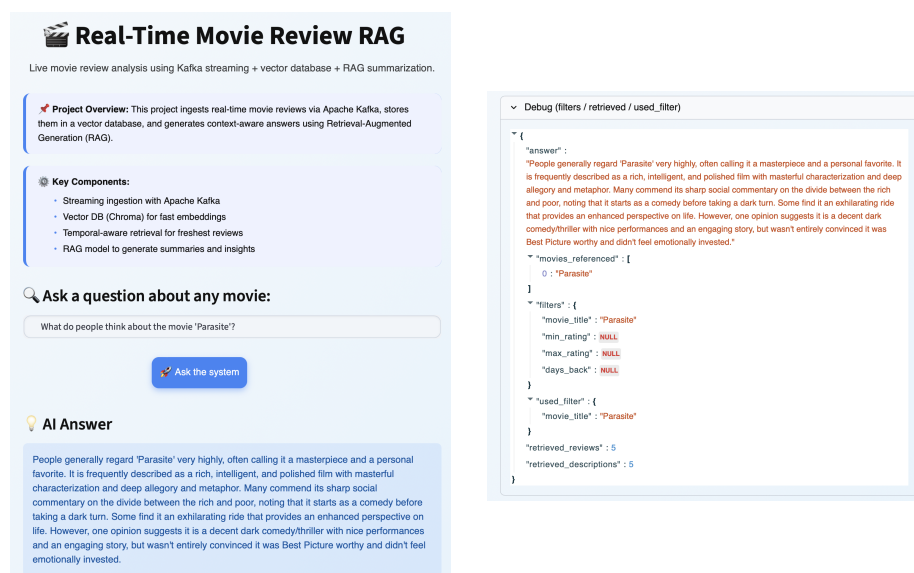


Figure 2: Example of a user prompt and the real-time RAG system’s generated answer.

Figure 3 shows a new example with different filters of this interaction, with the user prompt and the RAG-generated response.

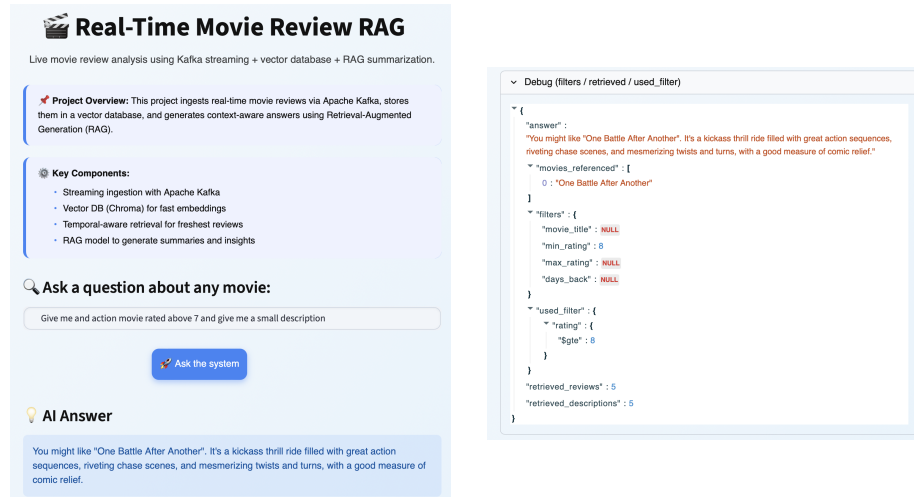


Figure 3: Another example of a user prompt and the real-time RAG system’s generated answer.

6 Conclusion

This project demonstrates the design and implementation of a **real-time Retrieval-Augmented Generation (RAG) system** for analyzing and summarizing live movie reviews. By integrating **Apache Kafka** for streaming ingestion, **ChromaDB** for fast semantic retrieval, and the **Gemini model** for embeddings and language generation, the system is able to provide up-to-date, context-aware answers to user queries.

The architecture ensures low-latency retrieval while maintaining relevance and freshness of information, highlighting how RAG techniques can be effectively applied to continuously evolving data streams.

Overall, this work illustrates the potential of combining streaming data pipelines with retrieval-augmented models, providing a scalable and interactive solution for real-time sentiment analysis and insights in the movie domain.